Project: Stack-Based LC-3 Calculator

1. Project Goal (What you're building)

Design and implement a **stack-based integer calculator** in LC-3 assembly. The deadline of this project is now set to **12/05, 23:50**.

Your calculator will:

- Read **commands and decimal integers** from the keyboard
- Use a **stack** in memory to store operands and intermediate results
- Perform **addition, subtraction, and multiplication** on **2's complement** integers
- Use **ASCII ↔ binary conversion** to interact with the user (keyboard/monitor)
- Display results on the console

This project is about more than "getting it to work": you should understand the **control flow**, **subroutine call structure**, and **data stack behavior** that make the calculator function.

---

2. Learning Objectives

By the end of this project, you should be able to:

1. Organize a non-trivial LC-3 program into **multiple subroutines** with clean call/return conventions.
2. Explain the difference between:
    - The **data stack** used by the calculator, and
    - The **call stack / register-saving discipline** used by subroutines.
3. Implement and trace:
    - **ASCII → 2's complement integer** conversion
    - **2's complement integer → ASCII** conversion
4. Use **TRAP routines** (`GETC`, `OUT`, `PUTS`, `HALT`) to build a text-based interface.
5. Reason about **control flow**: main loop, dispatch on commands, and subroutine call graphs.

---

3. Functional Requirements

Your LC-3 calculator must support at least the following:

3.1 Number Format and Range

- Accept **signed decimal integers** in the range **–999 to +999** (inclusive).
- Input is via **ASCII digits** on the keyboard.
- Internally, the calculator uses **16-bit 2's complement** for arithmetic, consistent with LC-3.

3.2 Commands

Your calculator must continuously prompt the user:

```
Enter a command:
```

The user types a key (or a sequence of digits + Enter). Support these commands:

- **X** – Exit the calculator (halt the program).
- **C** – Clear the stack (discard all stored values).
- **D** – Display the **value at the top of the stack**.
- **+** – Pop the top two values `A`, `B`, and push `A + B`.
- * – Pop the top two values `A`, `B`, and push `A * B`.
- **-** – Pop the top value `A` and push **–A** (unary minus).
- **[digits] + Enter** – Interpret the digits as a decimal integer and push it on the stack.
    - Example: typing 1, 7, 2, then Enter pushes 172.

Error-handling requirements:

- If the user:

    - Tries to push a non-integer string,
    - Enters more than three digits, or
    - Causes an arithmetic result outside the range –999..+999,

  your program must **print an error message** and **leave the stack in a consistent state**.

- If an operation needs more operands than exist on the stack (e.g., + when there's only one value), print an error and leave the stack unchanged.

---

4. Crucial Steps / Milestones

You can structure your work into the following milestones.

Step 1: Design the Memory Layout and Stack

- Choose a region of memory to use as a **stack**. For example:
    - `StackMax` – highest address used by the stack
    - `StackBase` – lowest address used by the stack

- Let **R6** act as the **stack pointer**:
  - Start R6 at `StackBase + 1` to represent an **empty** stack.
- Decide on a maximum stack depth (e.g., 10 values), and implement:
  - **PUSH** (check for overflow)
  - **POP** (check for underflow)

You will need routines like:

- `PUSH` – pushes the value in `R0` onto the stack; uses `R5` to signal success/failure.
- `POP` – pops the top of the stack into `R0`; uses `R5` to signal success/failure.

Step 2: Data Type Conversion Routines

Implement (or reuse) routines to:

1. **ASCII → binary** (`ASCIItoBinary`)
   - Input: an ASCII string of up to **three decimal digits** stored at some buffer `ASCIIBUFF`.
   - Output: a 2's complement integer in `R0`.
   - You'll need to:
     - Strip the ASCII template (`x0030` offset),
     - Compute hundreds, tens, and ones (e.g., using lookup tables or repeated additions).
2. **Binary → ASCII** (`BinarytoASCII`)
   - Input: `R0` holds a 2's complement integer in range –999..+999.
   - Output: four ASCII characters in memory starting at `ASCIIBUFF`:
     - sign (`'+'` or `'-'`)
     - hundreds digit
     - tens digit
     - ones digit

A simple strategy:

- Repeated subtraction of 100 for the hundreds digit,
- Repeated subtraction of 10 for the tens digit,
- Remaining value is the ones digit.

Step 3: Stack-Based Arithmetic Operations

Implement the following operator subroutines, each using the stack:

1. `OpAdd`
   - Pops two values from the stack, adds them, range-checks, and pushes the result.
   - If:
     - There are too few values, or
     - The result is out of range

     print an error and restore the stack to its original state.

2. `OpMult`
    - Pops two values, multiplies them using repeated addition (LC-3 has no `MUL` instruction).
    - Handle sign correctly.
    - Range-check the result before pushing.
3. `OpNeg`
    - Pops top value `A`, computes –`A`, pushes it.
4. `RangeCheck` (helper)
    - Verifies that the value in `R0` is in [–999, +999].
    - Sets a success/failure flag in `R5` and prints an error if out of range.

Step 4: Input Handling – Reading Numbers and Commands

Implement a `PushValue` subroutine that:

1. Reads characters using `GETC` and echoes them with `OUT`.
2. Collects up to **three digits** into `ASCIIBUFF`.
3. Terminates when the user presses **Line Feed / Enter**.
4. Validates that all characters were digits, and there was at least one digit:
    - If invalid: print an appropriate error message.
5. Counts how many digits were typed.
6. Calls `ASCIItoBinary` to convert the ASCII string to binary.
7. Calls `PUSH` to place the value on the stack.

This is where you see the **ASCII string ↔ binary integer** transition very concretely.

Step 5: Main Control Loop

Implement the main “calculator loop”:

1. Initialize:
    - `R6` (stack pointer),
    - Any global labels (`StackMax`, `StackBase`, `ASCIIBUFF`, etc.).
2. Loop:
    - Print `"Enter a command:"` using `PUTS`.
    - Read one character with `GETC`, echo it with `OUT`.
    - Dispatch based on the character:
        - If `'X'` – `HALT`
        - If `'C'` – call `OpClear` (reset stack pointer)
        - If `'+'` – call `OpAdd`
        - If `'*'` – call `OpMult`
        - If `'-'` – call `OpNeg`
        - If `'D'` – call `OpDisplay`:
            - `POP` the top value
            - Call `BinarytoASCII`
            - Print with `PUTS`
            - (Optionally) push the value back so the stack isn’t destroyed
        - Otherwise – treat the input as the **start of a number** and call `PushValue`.

3. After handling the command, go back to the prompt.

---

5. Hints and Tips

- **Use R6 for the stack pointer**, as is conventional in LC-3 examples.
- Maintain a **clear calling convention**:
  - Save any registers you modify in a subroutine (e.g., `R0`–`R3`, `R5`, `R7`) to local storage and restore before `RET`.
- Draw the **call graph**:
  - `Main → OpAdd → POP, RangeCheck, PUSH`
  - `Main → OpMult → POP, RangeCheck, PUSH`
  - `Main → OpDisplay → POP, BinarytoASCII, PUTS`
  - `Main → PushValue → ASCIItoBinary, PUSH`, etc.
- Separate the **data stack** (where you store calculator values) from the **ASCII buffer** (where you store characters from the user).
- Test incrementally:
  1. Verify `PUSH` / `POP`.
  2. Verify ASCII↔binary conversions with known values.
  3. Verify `OpAdd`, then `OpMult`, then `OpNeg`.
  4. Finally, plug everything into the main loop.

---

6. Deliverables

1. **LC-3 Assembly Source File(s)**
   - All code needed to assemble and run your calculator.
2. **Short Design Report (1–3 pages)** – Bonus points
   - Description of:
     - Memory layout (stack region, buffer region)
     - Major subroutines and their roles
     - Call graph diagram
   - Explanation of:
     - How you handle errors
     - Limitations of your design
3. **Design Artifacts (Bonus)**
   - Block diagrams, state diagrams, or stack diagrams showing:
     - How commands flow through the system
     - How the stack changes for an example expression
4. **Test Plan / Transcript (Optional/Bonus)**
   - Example sequences of inputs and outputs showing:
     - Normal operations
     - Error cases (overflow, underflow, invalid input)

---

7. Grading Criteria

1. **Correctness (40%)**
   - Calculator accepts and correctly processes:
     - Numbers in range –999..+999

        - Commands `X`, `C`, `D`, `+`, `*`, `-`
    - Stack operations behave correctly, including edge cases.
    - Arithmetic results are correct and properly range-checked.
2. **Robustness & Error Handling (15%)**
    - Handles:
        - Too many digits
        - Non-digit characters in numeric input
        - Stack underflow on `POP`/operations
        - Stack overflow on `PUSH`
    - Leaves stack consistent after errors.
3. **Code Organization & Style (20%)**
    - Meaningful labels and comments
    - Clear division into subroutines
    - Consistent register-saving conventions
4. **Understanding & Explanation (15%)**
    - Ability to explain:
        - Control flow in the main loop
        - How the stack is used for operands
        - How ASCII ↔ binary conversion works
        - How `RangeCheck` and error messages integrate
5. **Bonus (up to +10%)**
    - Design report with diagrams (call graph, stack evolution)
    - Additional features:
        - Allow negative input numbers directly (e.g., typing `-1 7 2` then Enter)
        - Cleaner output formatting (no unnecessary leading zeros or plus sign)
        - Additional operations (if implemented correctly and documented)